



**Dr. D. Y. Patil Institute of Technology
Pimpri Pune-411018**

Department of Artificial Intelligence and Data Science

Laboratory Manual

Savitribai Phule Pune University

Second Year of Artificial Intelligence and Data Science (2020 Course)

Subject Code: 217522

Data Structures Laboratory

Prepared by

1. Prof. Sandeep Chitalkar

Academic Year

2023-2024



**Dr. D. Y. Patil Institute of Technology
Pimpri Pune-411018**

Vision and Mission of Institute

Vision of the Institute

- Empowerment through knowledge

Mission of the Institute

Developing human potential to serve the Nation

- Dedicated efforts for quality education.
- Yearning to promote research and development.
- Persistent endeavor to imbibe moral and professional ethics.
- Inculcating the concept of emotional intelligence.
- Emphasizing extension work to reach out to the society.
- Treading the path to meet the future challenges.



**Dr. D. Y. Patil Institute of Technology
Pimpri Pune-411018**

Vision and Mission
Department of Artificial Intelligence and Data Science

Vision of the Department

- To produce globally competent engineers in the field of Artificial Intelligence and Data Science with human values

Mission of the Department

- To develop students with a sound understanding in the area of Artificial Intelligence, Machine Learning and Data Science.
- To enable students to become innovators, researchers, entrepreneurs and leaders globally.
- Equip the department with new advancement in high performance equipments and software to carrying out research in emerging technologies in AI and DS.
- To meet the pressing demands of the nation in the areas of Artificial Intelligence and Data Science.

Data Structures Laboratory

217522

Teaching Scheme	Credit Scheme	Examination Scheme and Marks
Practical: 04 Hours/Week	02	Term Work: 25 Marks Practical: 50 Marks

Course Objectives:

To understand basic techniques and strategies of algorithm analysis, the memory requirement for various data structures like array, linked list, stack, queue etc using concepts of python and C++ programming language.

Course Outcomes:

On completion of the course, learner will be able to–

C01: Use algorithms on various linear data structure using sequential organization to solve real life problems.

C02: Analyze problems to apply suitable searching and sorting algorithm to various applications.

C03: Analyze problems to use variants of linked list and solve various real life problems.

C04: Designing and implement data structures and algorithms for solving different kinds of problems.

Guidelines for Instructor's Manual

The instructor's manual is to be developed as a hands-on resource and reference. The instructor's manual need to include prologue (about University/program/ institute/ department/foreword/ preface), curriculum of course, conduction and Assessment guidelines, topics under consideration- concept, objectives, outcomes, set of typical applications/assignments/ guidelines, and references.

Guidelines for Student's Laboratory Journal

The laboratory assignments are to be submitted by student in the form of journal. Journal consists of prologue, Certificate, table of contents, and handwritten write-up of each assignment (Title, Objectives, Problem Statement, Outcomes, software and Hardware requirements, Date of Completion, Assessment grade/marks and assessor's sign, Theory- Concept in brief, algorithm, flowchart, test cases, Test Data Set(if applicable), mathematical model (if applicable), conclusion/analysis. Program codes with sample output of all performed assignments are to be submitted as softcopy.

As a conscious effort and little contribution towards Green IT and environment awareness, attaching printed papers as part of write-ups and program listing to journal may be avoided. Use of DVD containing students programs maintained by Laboratory In-charge is highly encouraged. For reference one or two journals may be maintained with program prints at Laboratory.

Guidelines for Laboratory / Term Work Assessment

Continuous assessment of laboratory work should be done based on overall performance and Laboratory assignments performance of student. Each Laboratory assignment assessment should be assigned grade/marks based on parameters with appropriate weightage. Suggested parameters for overall assessment as well as each Laboratory assignment assessment include- timely completion, performance, innovation, efficient codes, punctuality and neatness.

Guidelines for Laboratory Conduction

The instructor is expected to frame the assignments by understanding the prerequisites, technological aspects, utility and recent trends related to the topic. The assignment framing policy need to address the average students and inclusive of an element to attract and promote the intelligent students. The instructor may set multiple sets of assignments and distribute among batches of students. It is appreciated if the assignments are based on real world problems/applications. Encourage students for appropriate use of Hungarian notation, proper indentation and comments. Use of open source software is to be encouraged. In addition to these, instructor may assign one real life application in the form of a mini-project based on the concepts learned. Instructor may also set one assignment or mini-project that is suitable to respective branch beyond the scope of syllabus.

Set of suggested assignment list is provided in groups- A, B, C, D, and E. Each student must perform at least 13 assignments (at least 3 from group A, 3 from group B, 2 from group C, 2 from group D and 3 from group E.)

Group A and B assignments should be implemented in Python without using built-in methods for major functionality of assignment. Use List data structure of Python as array. Group C, D and E assignments should be implemented in C++ language.

Operating System recommended:- 64-bit Open source Linux or its derivative **Programming tools recommended:-** Open Source Python, Programming tool like Jupyter Notebook, Pycharm, Spyder, G++/GCC.

Virtual Laboratory:

<http://cse01-iiith.vlabs.ac.in/Courses%20Aligned.html?domain=Computer%20Science>

Practical No.	Assignment to be covered
3 Assignments from Group A using python	
1	Write a Python program to store marks scored in subject “Fundamental of Data Structure” by N students in the class. Write functions to compute following: a)The average score of class b)Highest score and lowest score of class c)Count of students who were absent for the test d)Display mark with highest frequency
2	Write a Python program to compute following operations on String: a)To display word with the longest length b)To determines the frequency of occurrence of particular character in the string c)To check whether given string is palindrome or not d)To display index of first appearance of the substring e)To count the occurrences of each word in a given string
3	Write a Python program to compute following computation on matrix: a)Addition of two matrices B) Subtraction of two matrices c) Multiplication of two matrices d) Transpose of a matrix
3 Assignments from Group B using python	
4	Write a Python program to maintain club members, sort on roll numbers in ascending order. Write function “Ternary_Search” to search whether particular student is member of club or not. Ternary search is modified binary search that divides array into 3 halves instead of two.
5	Write a Python program to store first year percentage of students in array. Write function for sorting array of floating point numbers in ascending order using a) Selection Sort b) Bubble sort and display top five scores.
6	Write a Python program to store first year percentage of students in array. Write function for sorting array of floating point numbers in ascending order using quick sort and display top five scores
2 Assignments from Group C using c++	
7	Department of Computer Engineering has student's club named 'Pinnacle Club'. Students of second, third and final year of department can be granted membership on request. Similarly one may cancel the membership of club. First node is reserved for president of club and last node is reserved for secretary of club. Write C++ program to maintain club member's information using singly linked list. Store student PRN and Name. Write functions to: a)Add and delete the members as well as president or even secretary. b)Compute total number of members of club c)Display members d)Two linked lists exists for two divisions. Concatenate two lists.
8	Write C++ program for storing binary number using doubly linked lists. Write functions a)To compute 1's and 2's complement b)Add two binary numbers
2 Assignments from Group D using c++	

9	A palindrome is a string of character that's the same forward and backward. Typically, punctuation, capitalization, and spaces are ignored. For example, "Poor Dan is in a droop" is a palindrome, as can be seen by examining the characters "poor danisina droop" and observing that they are the same forward and backward. One way to check for a palindrome is to reverse the characters in the string and then compare with them the original-in a palindrome, the sequence will be identical. Write C++ program with functions- a)To print original string followed by reversed string using stack b)To check whether given string is palindrome or not
10	Implement C++ program for expression conversion as infix to postfix and its evaluation using stack based on given conditions: 1. Operands and operator, both must be single character. 2. Input Postfix expression must be in a desired format. 3. Only '+', '-', '*' and '/' operators are expected.
3 Assignments from Group E using c++	
11	Queues are frequently used in computer programming, and a typical example is the creation of a job queue by an operating system. If the operating system does not use priorities, then the jobs are processed in the order they enter the system. Write C++ program for simulating job queue. Write functions to add job and delete job from queue
12	A double-ended queue (deque) is a linear list in which additions and deletions may be made at either end. Obtain a data representation mapping a deque into a one-dimensional array. Write C++ program to simulate deque with functions to add and delete elements from either end of the deque.
13	Pizza parlor accepting maximum M orders. Orders are served in first come first served basis. Order once placed cannot be cancelled. Write C++ program to simulate the system using circular queue using array.

Group A

Assignment 1

Problem Statement:

Write a Python program to store marks scored in subject "Fundamental of Data Structure" by N students in the class. Write functions to compute following: a)The average score of class b)Highest score and lowest score of class c)Count of students who were absent for the test d)Display mark with highest frequency

Objective:

1. Understand linear storage of data using array data structure.
2. Create these linear structures called Arrays
3. Manipulate the elements in the array using python
4. Understand Creation of class, object and calling of function in python.

Outcome:

Input:

1. Accept marks scored for first test of subject ' Fundamental of Data Structures ' for N students.
2. Store all the values using linear data structure i.e. Array.
3. If student is absent place -1 in array.

Output:

1. Average score of class
2. Highest mark and lowest mark among N students
3. Total number of student who score Highest Mark.
4. Total number of absent students.

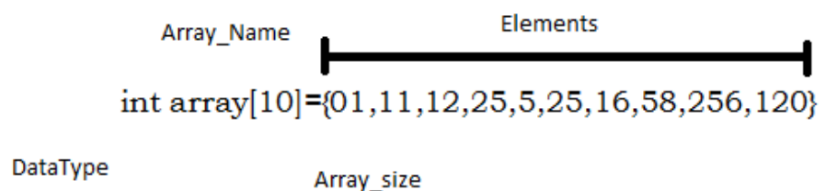
Theory:

What is Array?

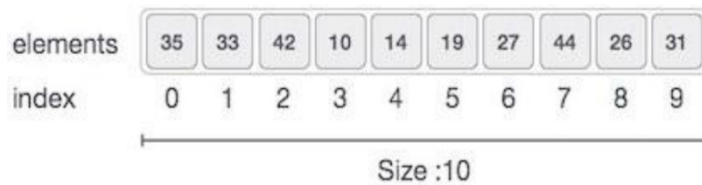
In computer science generally, a collection of data items that can be selected by indices computed at run-time, including: Array data structure, an arrangement of items at equally spaced addresses in computer memory. Array data type, used in a programming language to specify a variable that can be indexed.

Array Representation:

Arrays can be declared in various ways in different languages. For illustration, let's take C / C++ array declaration.



Arrays can be declared in various ways in different languages. For illustration, let's take C / C++ array declaration.



As per above shown illustration, following are the important points to be considered.

1. Index starts with 0.
2. Array length is 10 which mean it can store 10 elements from index 0 to 9.
3. Each element can be accessed via its index. For example, we can fetch element 27 at index 6.

Basic Operations:

Following are the basic operations supported by an array.

1. Traverse – print all the array elements one by one.
2. Insertion – add an element at given index.
3. Deletion – delete an element at given index.
4. Search – search an element using given index or by value.
5. Update – update an element at given index.

In python, LIST is a collection of arbitrary objects, somewhat akin to an array in many other programming languages but more flexible. Lists are defined in Python by enclosing a comma-separated sequence of objects in square brackets ([]), as shown below:

```
#create a list in python
```

```
>>> a = ['foo', 'bar', 'baz', 'qux']
```

```
#Display list in python
```

```
>>> print(a)
```

```
Output: ['foo', 'bar', 'baz', 'qux']
```

The important characteristics of Python lists are as follows:

1. Lists are ordered.
2. Lists can contain any arbitrary objects.
3. List elements can be accessed by index.
4. Lists can be nested to arbitrary depth.
5. Lists are mutable.
6. Lists are dynamic.

Lists Are Ordered:

A list is not merely a collection of objects. It is an ordered collection of objects. The order in which you specify the elements when you define a list is an innate characteristic of that list and is maintained for that list's lifetime.

Python has a set of built-in methods that you can use on lists/arrays.

Method	Description
<u>append()</u>	Adds an element at the end of the list
<u>clear()</u>	Removes all the elements from the list
<u>copy()</u>	Returns a copy of the list
<u>count()</u>	Returns the number of elements with the specified value
<u>extend()</u>	Add the elements of a list (or any iterable), to the end of the current list
<u>index()</u>	Returns the index of the first element with the specified value
<u>insert()</u>	Adds an element at the specified position
<u>pop()</u>	Removes the element at the specified position
<u>remove()</u>	Removes the first item with the specified value
<u>reverse()</u>	Reverses the order of the list
<u>sort()</u>	Sorts the list

Functions Used to implement problem statement: Write algorithm/pseudo code for each function.

1. The average score of class
2. Highest score and lowest score of class
3. Marks scored by most of the students
4. List of students who were absent for the test

Algorithm:

ALGORITHM : ACCEPT

1. Start
2. Read total number of students „n , maximum possible marks „max .
3. Initialize count[] to 0
4. Repeat till i = 0 to n,
 - a. Read rollno and marks
 - b. If marks = -1
 - i. Increment absent count by 1
 - c. Else
 - i. Increment count at the read marks by 1
5. Stop

ALGORITHM : AVERAGE

1. Repeat till i = 0 to n,
 - a. If marks[i] not equal to -1
 - i. sum = sum + marks[i]
2. Average = sum / (n – absent)
3. Stop

ALGORITHM : HIGHEST

1. Highest = -999
2. Repeat till i = 0 to n,
 - a. If marks[i] > highest,
 - i. Highest = marks[i]
3. Stop

ALGORITHM : LOWEST

1. Lowest = 999
2. Repeat till i = 0 to n,
 - a. If marks[i] < lowest
 - i. Lowest = marks[i]
3. Stop

ALGORITHM : MOSTMARKS

1. Mostcount = count [0]
2. Repeat till i = 1 to max,
 - a. If count[i] > mostcount
 - i. Mostcount = i
 - ii. Mostmarks = count [i]
3. Stop

ALGORITHM : DISPLAYMARKS

1. Start
2. Repeat till i = 0 to n
 - a. Display current roll no and marks
3. Stop

ALGORITHM : DISPLAYRESULT

1. Start
2. Display average, highest, lowest , mostcount and mostmarks
3. Repeat till i= 0 to n
 - a. If marks = -1
 - i. Display roll no as absent
4. Stop

ALGORITHM : MAIN

1. Start
2. Create object m
3. Call m.ACCEPT
4. Call m.DISPLAYMARKS
5. Call m.RESULTANALYSIS
6. Call m.DISPLAYRESULT
7. Stop

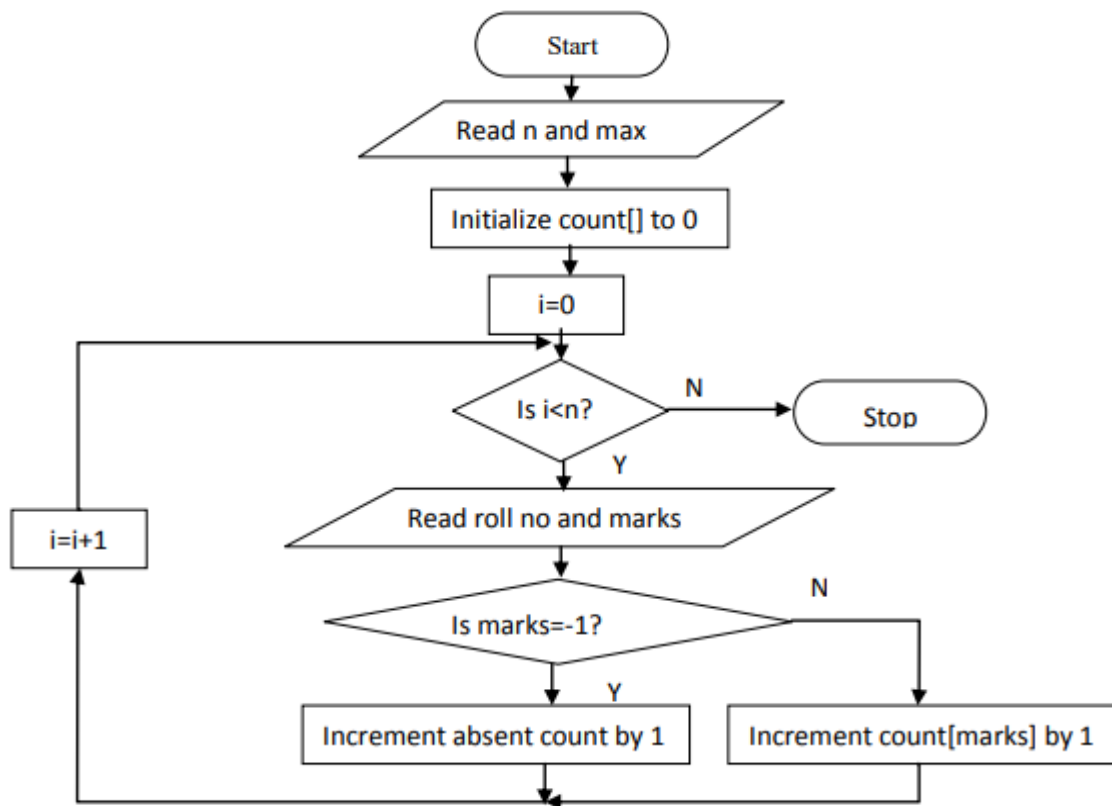
Conclusion:

By this way, we can store the marks of N students successfully.

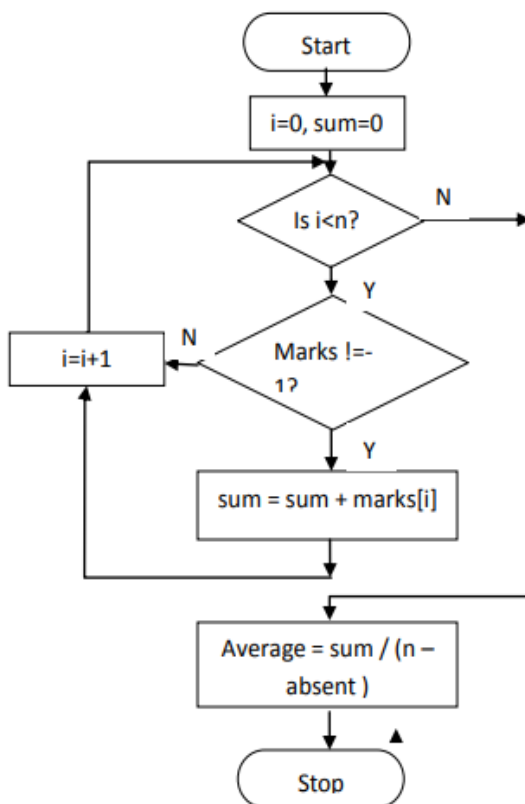
Oral Questions:

1. What is list python?
2. What are different methods are used in list?
3. What are functions?
4. What is class and object?
5. What is data abstraction?

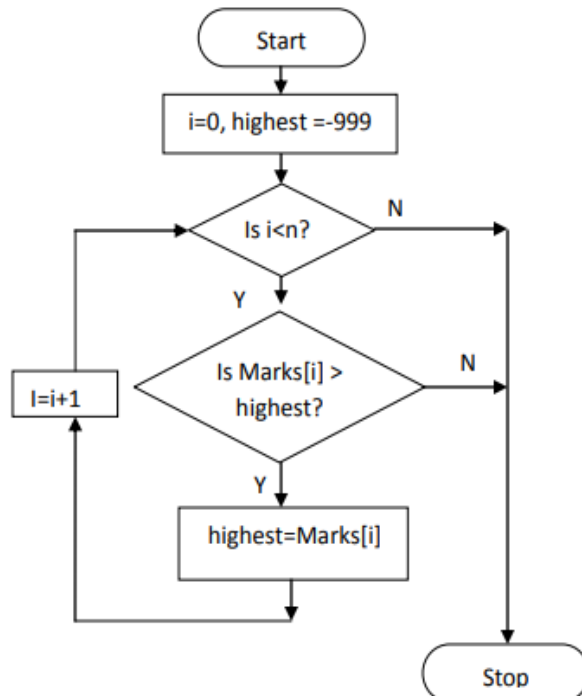
Flow chart to Accept



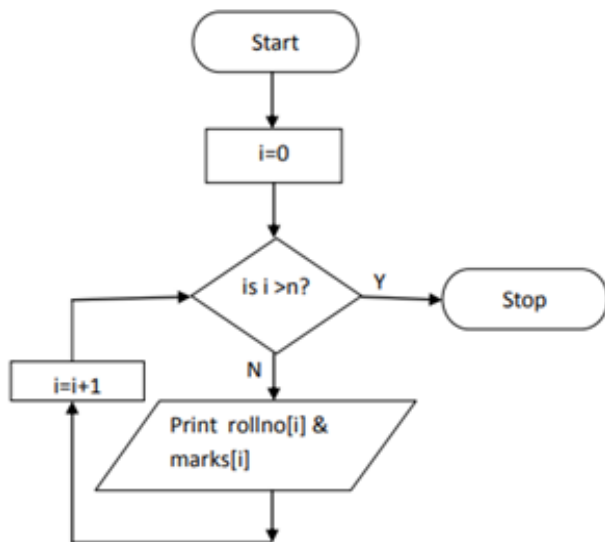
FLOWCHART : AVERAGE



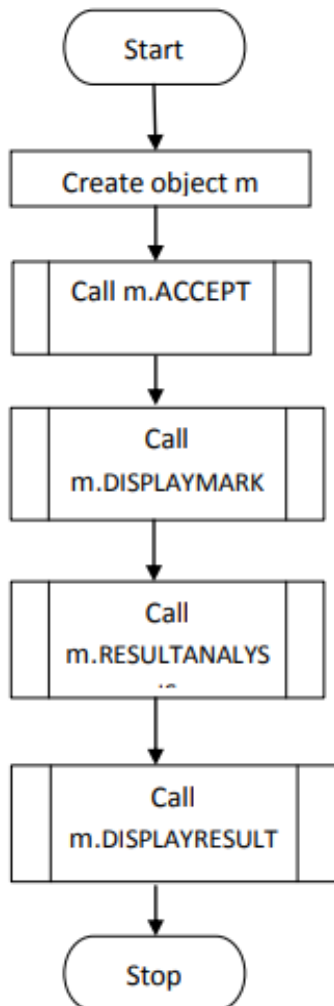
FLOWCHART: HIGHEST



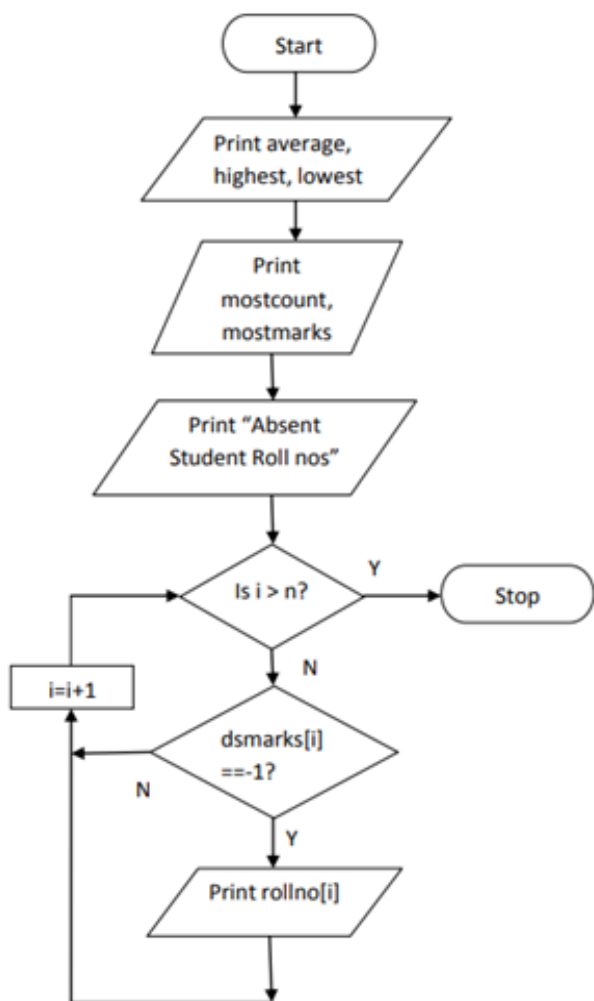
FLOWCHART : DISPLAYMARKS



FLOWCHART : MAIN



FLOWCHART : DISPLAYRESULT



Group A

Assignment 2

Problem Statement:

Write a Python program to compute following operations on String:

- a) To display word with the longest length
- b) To determines the frequency of occurrence of particular character in the string
- c) To check whether given string is palindrome or not
- d) To display index of first appearance of the substring
- e) To count the occurrences of each word in a given string

Objective:

1. Understand String manipulation operations using array and inbuilt library functions.
2. Create character array and store string in array.
3. String manipulation using array

Outcome

Input:

1. Accept String from user
2. Store the string using linear data structure i.e. Array.

Output:

1. Calculate length of longest word from String and display the word.
2. Calculate the frequency of occurrence of particular character in the string
3. Check whether given string is palindrome or not
4. Display index of first appearance of the substring
5. To count the occurrences of each word in a given string

Theory:

What is String in Python?

A string is a sequence of characters. A character is simply a symbol. For example, the English language has 26 characters. Computers do not deal with characters; they deal with numbers (binary). Even though you may see characters on your screen, internally it is stored and manipulated as a combination of 0s and 1s.

This conversion of character to a number is called encoding, and the reverse process is decoding. ASCII and Unicode are some of the popular encodings used. In Python, a string is a sequence of Unicode characters. Unicode was introduced to include every character in all languages and bring uniformity in encoding. You can learn about Unicode from Python Unicode.

Creating strings is as simple as assigning a value to a variable. For example –

```
var1 = 'Hello World!'
```

```
var2 = "Python Programming"
```

Note that there is no character data type in Python. A character is simply represented as a string with one character.

How to create a string in Python?

Strings can be created by enclosing characters inside a single quote or double-quotes. Even triple quotes can be used in Python but generally used to represent multiline strings and docstrings.

```
# defining strings in Python
# all of the following are equivalent
my_string = 'Hello'
print(my_string)

my_string = "Hello"
print(my_string)

my_string = '''Hello'''
print(my_string)

# triple quotes string can extend multiple lines
my_string = """Hello, welcome to
    the world of Python"""
print(my_string)
```

How to access characters in a string?

We can access individual characters using indexing and a range of characters using slicing. Index starts from 0. Trying to access a character out of index range will raise an `IndexError`. The index must be an integer. We can't use floats or other types, this will result into `TypeError`.

Python allows negative indexing for its sequences.

The index of -1 refers to the last item, -2 to the second last item and so on. We can access a range of items in a string by using the slicing operator `:`(colon).

#Accessing string characters in Python

```
str = 'programiz'
print('str = ', str)
```

```
#first character
print('str[0] = ', str[0])
```

```
#last character
print('str[-1] = ', str[-1])
```

```
#slicing 2nd to 5th character
print('str[1:5] = ', str[1:5])
```

```
#slicing 6th to 2nd last character
print('str[5:-2] = ', str[5:-2])
```

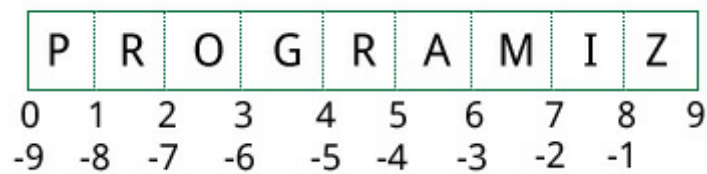
If we try to access an index out of the range or use numbers other than an integer, we will get errors.

```
# index must be in range
>>> my_string[15]
...
IndexError: string index out of range
```

```
# index must be an integer
>>> my_string[1.5]
...
TypeError: string indices must be integers
```

Slicing can be best visualized by considering the index to be between the elements as shown below.

If we want to access a range, we need the index that will slice the portion from the string.



ESCAPE CHARACTERS

As a side note, there are a number of escape characters supported by Python strings. The most common ones are:

- '\n' – newline
- '\s' – space
- '\t' – tab

Python has a set of built-in methods that you can use on strings.

Method	Description
<code>capitalize()</code>	Converts the first character to upper case
<code>casefold()</code>	Converts string into lower case
<code>center()</code>	Returns a centered string
<code>count()</code>	Returns the number of times a specified value occurs in a string
<code>encode()</code>	Returns an encoded version of the string
<code>endswith()</code>	Returns true if the string ends with the specified value
<code>expandtabs()</code>	Sets the tab size of the string
<code>find()</code>	Searches the string for a specified value and returns the position of where it was found
<code>format()</code>	Formats specified values in a string
<code>format_map()</code>	Formats specified values in a string
<code>index()</code>	Searches the string for a specified value and returns the position of where it was found
<code>isalnum()</code>	Returns True if all characters in the string are alphanumeric

<u>isalpha()</u>	Returns True if all characters in the string are in the alphabet
<u>isdecimal()</u>	Returns True if all characters in the string are decimals
<u>isdigit()</u>	Returns True if all characters in the string are digits
<u>isidentifier()</u>	Returns True if the string is an identifier
<u>islower()</u>	Returns True if all characters in the string are lower case
<u>isnumeric()</u>	Returns True if all characters in the string are numeric
<u>isprintable()</u>	Returns True if all characters in the string are printable
<u>isspace()</u>	Returns True if all characters in the string are whitespaces
<u>istitle()</u>	Returns True if the string follows the rules of a title
<u>isupper()</u>	Returns True if all characters in the string are upper case
<u>join()</u>	Joins the elements of an iterable to the end of the string
<u>ljust()</u>	Returns a left justified version of the string
<u>lower()</u>	Converts a string into lower case
<u>lstrip()</u>	Returns a left trim version of the string
<u>maketrans()</u>	Returns a translation table to be used in translations
<u>partition()</u>	Returns a tuple where the string is parted into three parts
<u>replace()</u>	Returns a string where a specified value is replaced with a specified value
<u>rfind()</u>	Searches the string for a specified value and returns the last position of where it was found
<u>rindex()</u>	Searches the string for a specified value and returns the last position of where it was found
<u>rjust()</u>	Returns a right justified version of the string
<u>rpartition()</u>	Returns a tuple where the string is parted into three parts
<u>rsplit()</u>	Splits the string at the specified separator, and returns a list
<u>rstrip()</u>	Returns a right trim version of the string
<u>split()</u>	Splits the string at the specified separator, and returns a list
<u>splitlines()</u>	Splits the string at line breaks and returns a list
<u>startswith()</u>	Returns true if the string starts with the specified value
<u>strip()</u>	Returns a trimmed version of the string
<u>swapcase()</u>	Swaps cases, lower case becomes upper case and vice versa
<u>title()</u>	Converts the first character of each word to upper case
<u>translate()</u>	Returns a translated string
<u>upper()</u>	Converts a string into upper case
<u>zfill()</u>	Fills the string with a specified number of 0 values at the beginning

Note: All string methods returns new values. They do not change the original string.

Algorithm:

1. Start
2. Accept the string from user and Calculate length of longest word from String and display the word.
3. Accept the string and calculate the frequency of occurrence of particular character in the string.
4. Accept the string and Check whether given string is palindrome or not
5. Accept the string and Display index of first appearance of the substring
6. Accept the string and count the occurrences of each word in a given string.
7. Stop.
- 8.

Conclusion:

By this way, we can perform string operation by accepting string from user..

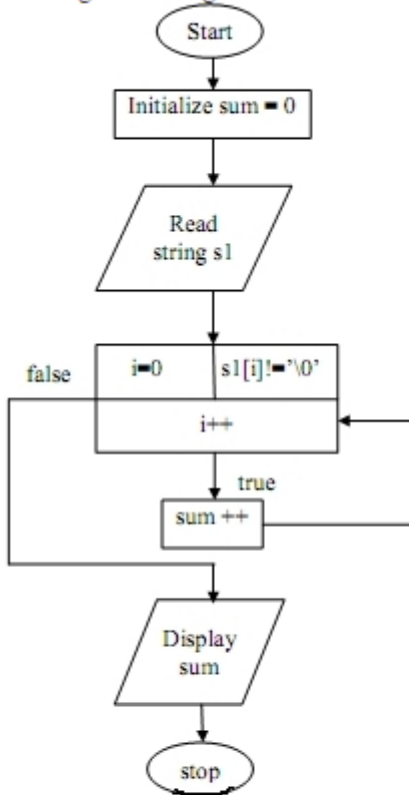
Oral Questions:

1. What is string in python?
2. What are different methods are used in string operation?
3. What is string palindrome?
4. What is slicing?
5. What is list? List is mutable or immutable?

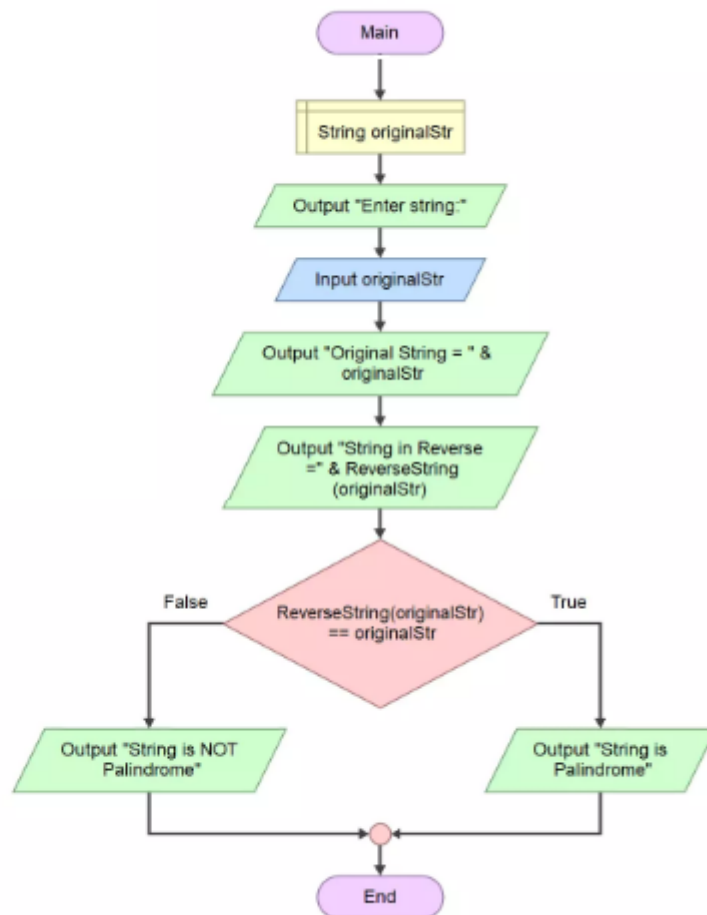
Flow Chart:

Draw the remaining flowcharts as per each operation

Length of String



String Palindrome



Group A

Assignment 3

Problem Statement:

Write a python program to compute following computation on matrix:

- Addition of two matrices
- Subtraction of two matrices
- Multiplication of two matrices
- Transpose of a matrix

OBJECTIVE:

- Understand matrix operation
- Understand the use of 2d array

OUTCOME:

Input:

- Accept matrix from user
- Store the matrix using linear data structure i.e. 2D Array.

Output:

- Addition of two matrices
- Subtraction of two matrices
- Multiplication of two matrices
- Transpose of a matrix

THEORY:

What is 2D array?

It is a collection of data elements of same data type arranged in rows and columns (that is, in two dimensions).

Declaration of Two-Dimensional Array in c++

```
DataTypearrayName[numberOfRows][numberOfColumn];
```

For example,

```
int Sales[3][5];
```

It creates a 2D array with 3 rows and 5 columns

	Col1	Col2	Col3	Col4	Col5
Row1					
Row2					
Row3					

Referring to Array Elements

To access the elements of a two-dimensional array, we need a pair of indices: one for the row position and one for the column position. The format is as simple as:

```
name[rowIndex][columnIndex];
```

Examples:

```
cout<<A[1][2]; //print an array element
A[1][2]=13; // assign value to an array element
cin>>A[1][2]; //input element
```

Using Loop to input an Two-Dimensional Array from user

```
int mat[3][5], row, col ;
for (row = 0; row < 3; row++)
    for (col = 0; col < 5; col++)
        cin>> mat[row][col];
```

It accept matrix named as mat[3][5] from user.

Algorithm:

1. Start
2. Accept the 2 matrix from user.
3. Perform addition of two matrices
4. Perform Subtraction of two matrices
5. Perform Multiplication of two matrices
6. Perform Transpose of matrix
7. Stop.

Conclusion:

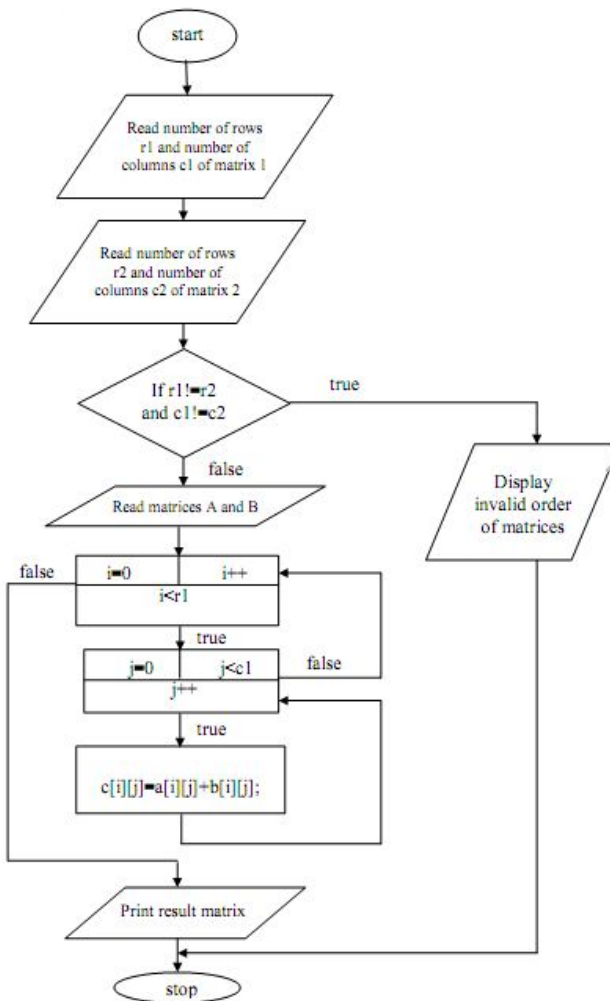
By this way, we can perform matrix operation by accepting matrix from user..

Questions:

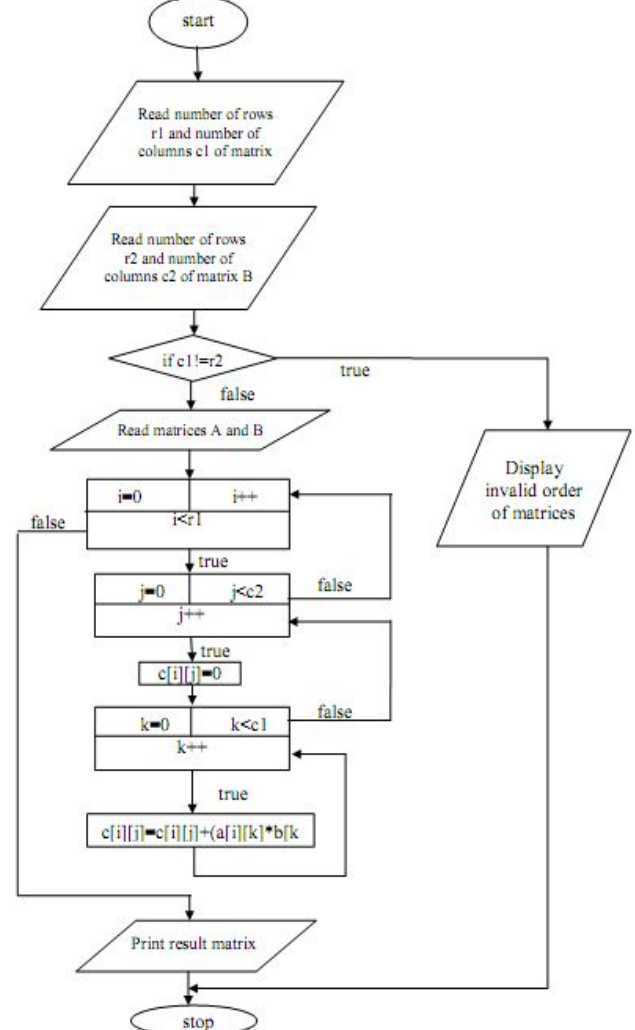
1. What is use of 2D array?
2. How to represent matrix using row major and column major order?

Flow chart:

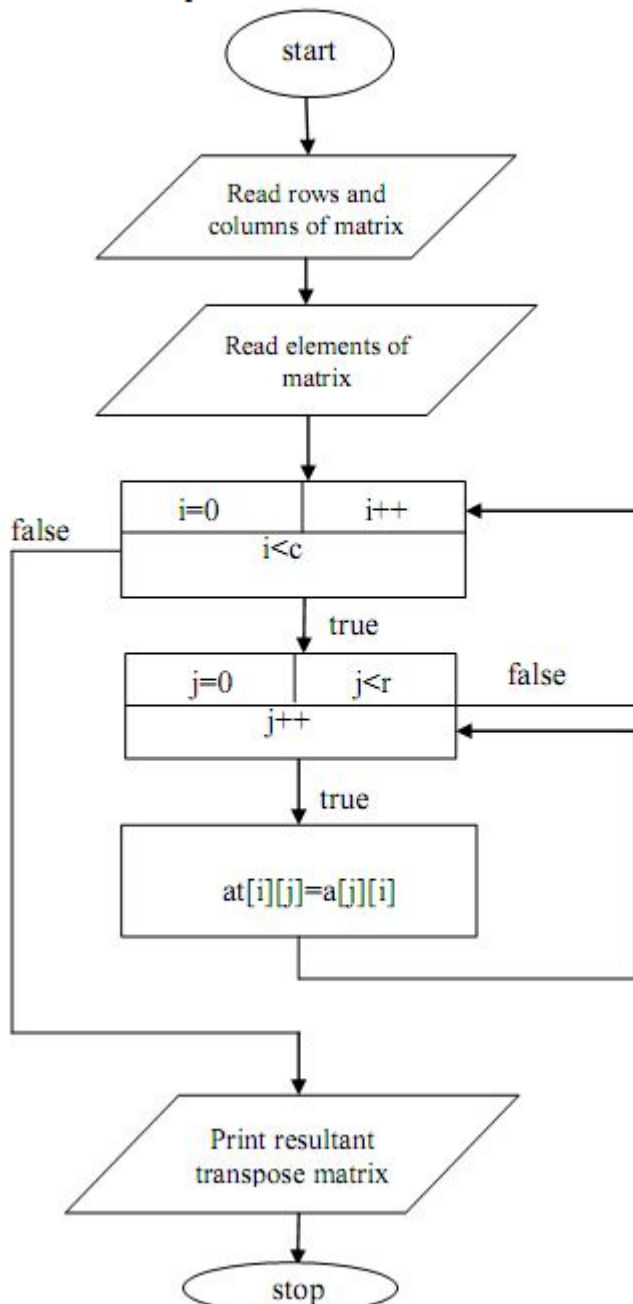
Addition of two matrix



Multiplication of two matrix



Transpose of matrix



Group B
Assignment 4

Problem Statement:

Write a Python program to maintain club members, sort on roll numbers in ascending order. Write function "Ternary_Search" to search whether particular student is member of club or not. Ternary search is modified binary search that divides array into 3 halves instead of two.

OBJECTIVE:-

1. To understand concept of searching.
2. To study the different searching methods.
3. To study the applications of each method.

OUTCOME:-

Input:

1. Read total size of the array from user
2. Read sorted element from user
3. Read the number to be searched in array

Output:

1. Tell whether element is present in array or not

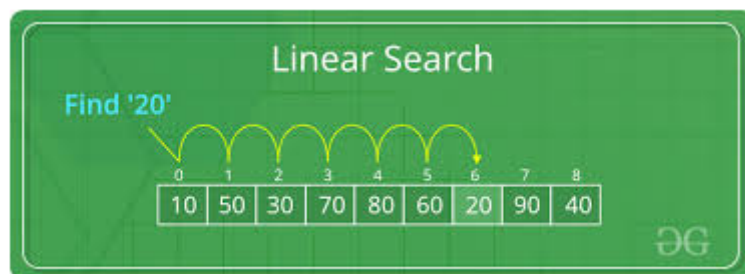
Theory

Searching, in computer science and information retrieval, refers to the process of finding a specific target or item within a collection of data or a data structure. The data structure could be an array, a list, a tree, a graph, a database, or any other organized collection of elements.

Here are some common types of searching:

1. Linear Search:

Linear search is a simple searching algorithm applicable to unsorted lists or arrays. It sequentially checks each element in the list until the target element is found or the end of the list is reached. Linear search has a time complexity of $O(n)$, where n is the number of elements in the list.

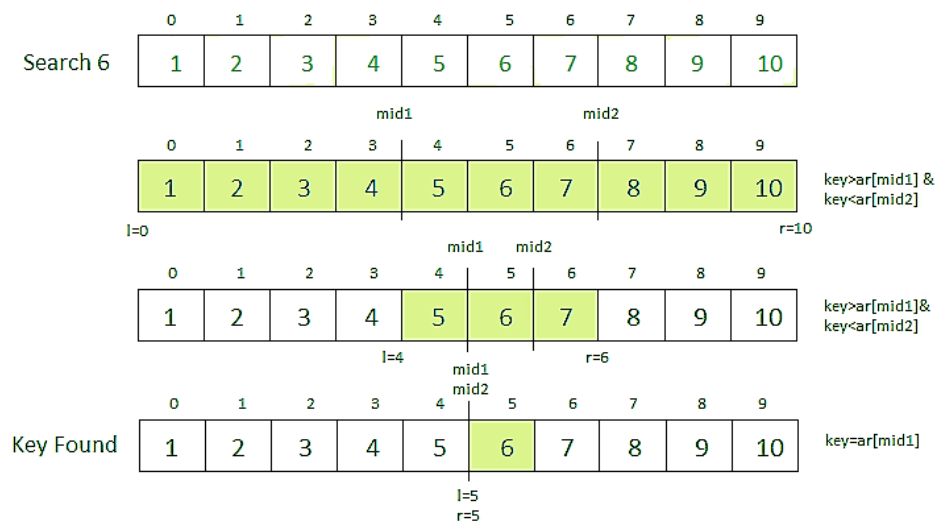


2. Binary Search:

Binary search is an efficient searching algorithm used for sorted lists or arrays. It repeatedly divides the search interval in half, efficiently narrowing down the search space to the desired element. Binary search has a time complexity of $O(\log n)$, where n is the number of elements in the list. It is significantly faster than linear search for large datasets.



3. Ternary Search: Ternary search is a divide-and-conquer algorithm used to find the minimum or maximum value of a unimodal function within a specified range. It divides the search space into three parts at each step, narrowing down the range until the desired precision is achieved.



Algorithm:-

1. Algorithm for Generating Linear Search:

Input: numbers provided for searching

Output: location of given number from list

Step1: Start

Step2: take input and store

Step3: take number to search

Step4: repeat until (n-1) times

Step5: check if number available in array

Step6: Print location of number

Step6: Else number is not present

Step7: End

2. Algorithm for Generating Binary Search:

Step 1 - Read the search element from the user.

Step 2 - Find the middle element in the sorted list.

Step 3 - Compare the search element with the middle element in the sorted list.

Step 4 - If both are matched, then display "Given element is found!!!" and terminate the function.

Step 5 - If both are not matched, then check whether the search element is smaller or larger than the middle element.

Step 6 - If the search element is smaller than middle element, repeat steps 2, 3, 4 and 5 for the left sublist of the middle element.

Step 7 - If the search element is larger than middle element, repeat steps 2, 3, 4 and 5 for the right sublist of the middle element.

Step 8 - Repeat the same process until we find the search element in the list or until sublist contains only one element.

Step 9 - If that element also doesn't match with the search element, then display "Element is not found in the list!!!" and terminate the function.

3. Algorithm for Generating Ternary Search:

The steps involved in this algorithm are:

(The list must be in sorted order)

Input: numbers provided for searching

Output: location of given number from list

Step 1- Divide the search space (initially, the list) in three parts (with two mid-points: mid1 and mid2)

Step 2- The target element is compared with the edge elements that is elements at location mid1, mid2 and the end of the search space.

Step 3- If element matches, go to step 3 else predict in which section the target element lies. The search space is reduced to 1/3rd.

Step 4- If the element is not in the list, go to step 6 or to step 1.

Step 5- Element found. Return index and exit.

Step 6: Element not found. Exit.

Complexity:

1. Linear Search Time complexity = $O(n)$.
2. Binary Search Time Complexity = $O(\log n)$.
3. Ternary Search Time Complexity = $O(\log_3 n)$, where n is the size of the array.

Conclusion:

Each searching algorithm has its strengths and weaknesses, and the choice of algorithm depends on factors like the size of the dataset, the type of data structure, and the specific requirements of the search operation.

Oral Questions:

1. What is the time complexity of the ternary search
2. What is ternary search
3. Explain what is a recursive algorithm
4. What is the time complexity of binary search
5. What is the time complexity of ternary search
6. How much comparison needed in best case searching.

Group B

Assignment 5

Problem Statement:

Write a Python program to store first year percentage of students in array. Write function for sorting array of floating point numbers in ascending order using

1. Selection Sort
2. Bubble sort and display top five scores.

OBJECTIVES:

1. To understand concept of sorting.
2. To study the different sorting techniques.
3. To study the applications of each method.

OUTCOME:-

Input:

1. Read total number of element want to store in array
2. Read data from user

Output:

1. Print sorted list using selection sort or bubble sort

Theory:

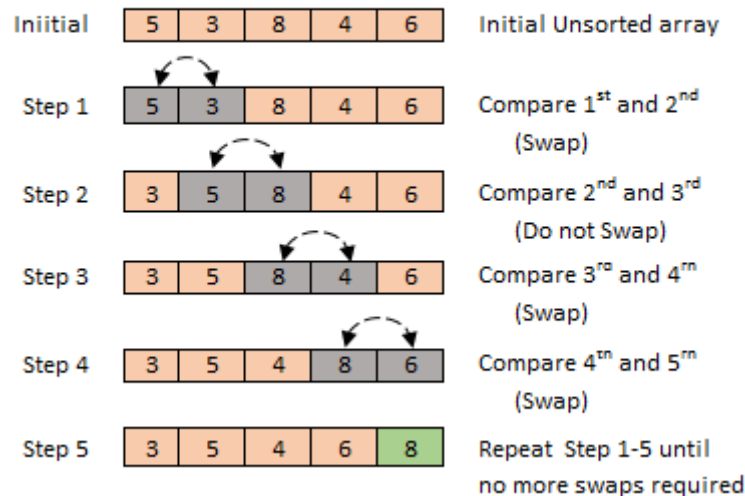
Sorting, in computer science, refers to the process of arranging a collection of data elements in a specific order, usually based on their key values. The primary goal of sorting is to bring organization to the data, making it easier and more efficient to search, retrieve, and process information. Sorted data allows for faster searching algorithms like binary search, and it often simplifies other data processing tasks.

There are numerous sorting algorithms available, each with its own set of advantages and disadvantages in terms of time complexity, space complexity, and stability. Some of the commonly used sorting algorithms include: Bubble Sort, Selection Sort, Insertion Sort, Merge Sort, Quick Sort, Heap Sort, Radix Sort, etc..

Bubble Sort:

Bubble Sort is a simple comparison-based sorting algorithm that repeatedly steps through the list to be sorted, compares adjacent elements, and swaps them if they are in the wrong order. The algorithm gets its name from the way smaller elements "bubble" to the top of the list as they get sorted.

Bubble Sort has a time complexity of $O(n^2)$ in the worst and average cases, where n is the number of elements in the list. It is not considered very efficient, especially for large datasets, as it takes a quadratic number of comparisons and swaps to sort the list.

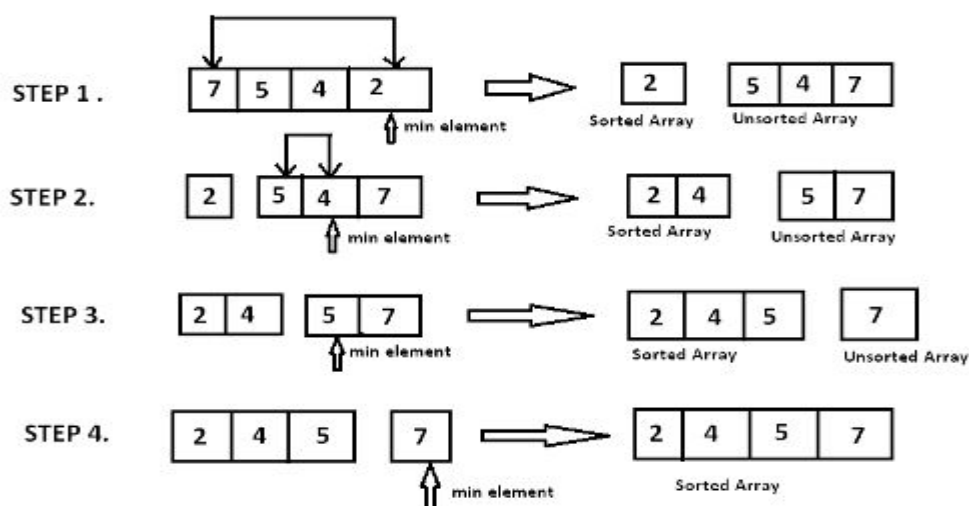


Here's how the Bubble Sort algorithm works:

1. Start with an unsorted list of elements.
2. Compare the first two elements. If the first element is greater than the second element, swap them.
3. Move to the next pair of elements (elements at index 1 and 2), and continue comparing and swapping until you reach the end of the list.
4. Now, the largest element is at the end of the list. Repeat steps 2 and 3, but this time, ignore the last element (as it's already in its correct position).
5. Continue this process, ignoring one more element from the end with each iteration, until the entire list is sorted.

Selection Sort:

Selection Sort is a simple comparison-based sorting algorithm that divides the input list into two parts: a sorted part and an unsorted part. The algorithm repeatedly finds the minimum (or maximum) element from the unsorted part and moves it to the end of the sorted part. This process is repeated until the entire list becomes sorted.



Here's how the Selection Sort algorithm works:

1. Start with an unsorted list of elements.
2. Set the "current index" as the first element of the list.
3. Find the minimum (or maximum) element from the unsorted part of the list and swap it with the element at the current index.
4. Move the "current index" to the next element and repeat step 3 for the remaining unsorted part.
5. Continue this process until the entire list is sorted.

Selection Sort has a time complexity of $O(n^2)$ in the worst, best, and average cases, where n is the number of elements in the list. It performs the same number of comparisons regardless of whether the list is already sorted or not. Additionally, it has a property of making only $O(1)$ swaps, which can be beneficial in situations where swapping elements is costly (e.g., for large data structures). However, Selection Sort is not considered efficient for large datasets due to its quadratic time complexity, and it is generally outperformed by more advanced sorting algorithms like Merge Sort, Quick Sort, or Heap Sort.

Conclusion:-

The choice of sorting algorithm depends on factors like the size of the dataset, the type of data structure, and the specific requirements of the sorting task. Some algorithms are more suitable for small datasets, while others are more efficient for large datasets.

Oral Questions:

1. How Bubble Sort Works?
2. How Selection Sort Works?
3. Time Complexity of Bubble sort and selection Sort?
4. Difference between Bubble sort and Selection sort?

Flow Chart: Draw the flowchart of bubble sort and selection sort.

Group B

Assignment 6

Problem Statement:

Write a Python program to store first year percentage of students in array. Write function for sorting array of floating point numbers in ascending order using Quick Sort.

OBJECTIVES:

1. To understand concept of Quick Sort.
2. To study the Quick Sort technique.
3. To study the application Quick Sort method.

OUTCOME:-

Input:

1. Read total number of element want to store in array
2. Read data from user

Output:

1. Print sorted list using selection sort or Quick sort

Theory:

Quicksort is a sorting algorithm based on the divide and conquer approach where

1. An array is divided into subarrays by selecting a pivot element (element selected from the array).
While dividing the array, the pivot element should be positioned in such a way that elements less than pivot are kept on the left side and elements greater than pivot are on the right side of the pivot.
2. The left and right subarrays are also divided using the same approach. This process continues until each subarray contains a single element.
3. At this point, elements are already sorted. Finally, elements are combined to form a sorted array.

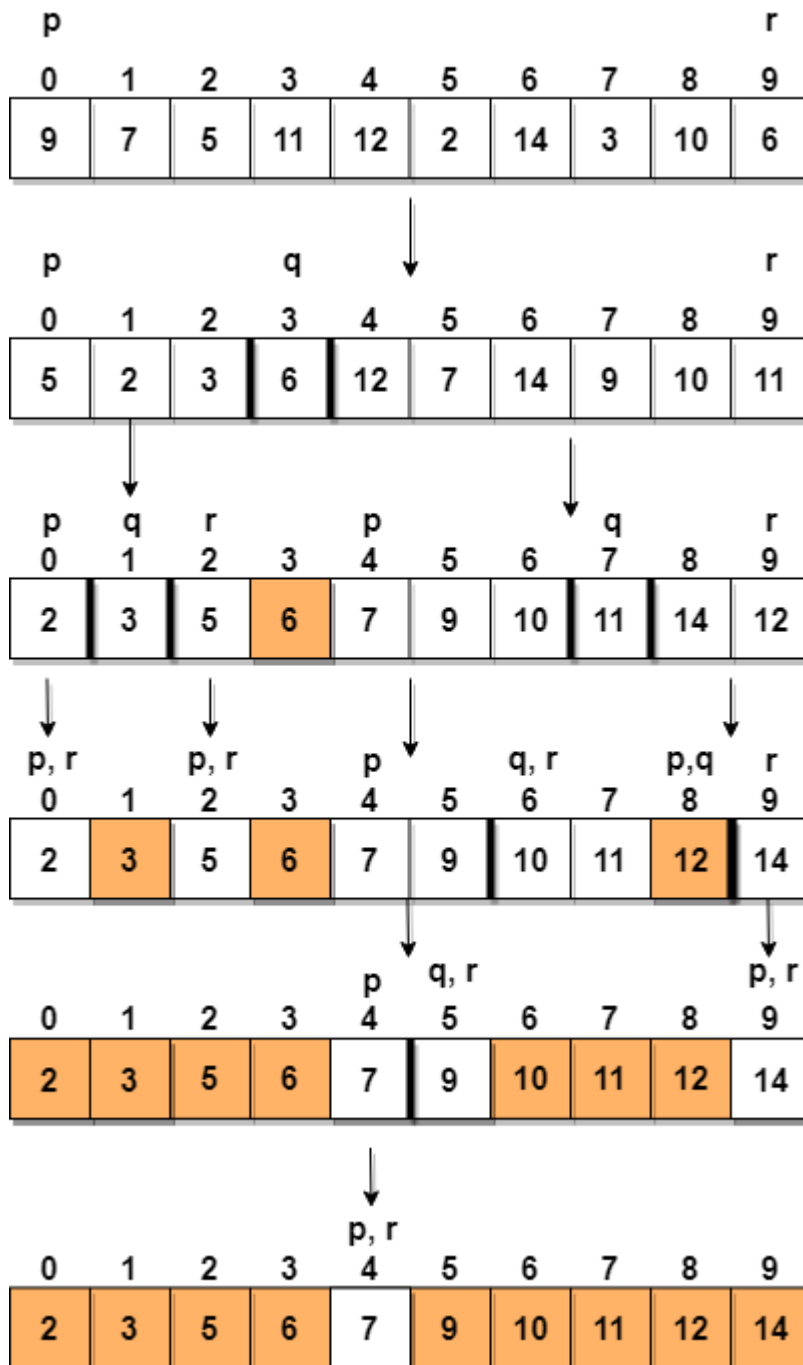
Here's a simplified explanation of the Quick sort algorithm:

1. After selecting an element as pivot, which is the last index of the array in our case, we divide the array for the first time.
2. In quick sort, we call this partitioning. It is not simple breaking down of array into 2 subarrays, but in case of partitioning, the array elements are so positioned that all the elements smaller than the pivot will be on the left side of the pivot and all the elements greater than the pivot will be on the right side of it.
3. And the pivot element will be at its final sorted position.
4. The elements to the left and right, may not be sorted.

- Then we pick subarrays, elements on the left of pivot and elements on the right of pivot, and we perform partitioning on them by choosing a pivot in the subarrays.

Let's consider an array with values {9, 7, 5, 11, 12, 2, 14, 3, 10, 6}

Below, we have a pictorial representation of how quick sort will sort the given array.



In step 1, we select the last element as the pivot, which is 6 in this case, and call for partitioning, hence re-arranging the array in such a way that 6 will be placed in its final position and to its left will be all the elements less than it and to its right, we will have all the elements greater than it.

Then we pick the subarray on the left and the subarray on the right and select a pivot for them, in the above diagram, we chose 3 as pivot for the left subarray and 11 as pivot for the right subarray.

And we again call for partitioning.

Complexity Analysis of Quick Sort

For an array, in which partitioning leads to unbalanced subarrays, to an extent where on the left side there are no elements, with all the elements greater than the pivot, hence on the right side.

And if keep on getting unbalanced subarrays, then the running time is the worst case, which is $O(n^2)$ Where as if partitioning leads to almost equal subarrays, then the running time is the best, with time complexity as $O(n \log n)$.

Worst Case Time Complexity [Big-O]: $O(n^2)$

Best Case Time Complexity [Big-omega]: $O(n \log n)$

Average Time Complexity [Big-theta]: $O(n \log n)$

Space Complexity: $O(n \log n)$

Algorithm:

```
quickSort(array, leftmostIndex, rightmostIndex)
  if (leftmostIndex < rightmostIndex)
    pivotIndex <- partition(array, leftmostIndex, rightmostIndex)
    quickSort(array, leftmostIndex, pivotIndex - 1)
    quickSort(array, pivotIndex, rightmostIndex)
```

```
partition(array, leftmostIndex, rightmostIndex)
  set rightmostIndex as pivotIndex
  storeIndex <- leftmostIndex - 1
  for i <- leftmostIndex + 1 to rightmostIndex
    if element[i] < pivotElement
      swap element[i] and element[storeIndex]
      storeIndex++
  swap pivotElement and element[storeIndex+1]
  return storeIndex + 1
```

Conclusion: Quick sort can be optimized with various techniques, like choosing a good pivot, using insertion sort for small sublists, etc.

Oral Question:

1. What is divide and conquer approach
2. What is worst case time complexity of quick sort
3. How is the pivot element chosen in the Quick sort algorithm, and why is its selection crucial for the efficiency of the algorithm?
4. Discuss the advantages and disadvantages of Quick sort

Group C

Assignment 7

Problem Statement:

Department of Computer Engineering has student's club named 'Pinnacle Club'. Students of Second, third and final year of department can be granted membership on request. Similarly one may cancel the membership of club. First node is reserved for president of club and last node is reserved for secretary of club. Write C++ program to maintain club member's information using singly linked list. Store student PRN and Name. Write functions to

- a) Add and delete the members as well as president or even secretary.
- b) Compute total number of members of club
- c) Display members
- d) Display list in reverse order using recursion
- e) Two linked lists exists for two divisions. Concatenate two lists.

OBJECTIVES:

1. To understand concept of Linked List.
2. To study the ADT of Singly linked list.

Input:

1. Accept Student name and student PRN number
2. First node is reserved for president of club and last node is reserved for secretary of club.

Output:

1. Display the members of the club (singly listed list).
2. Display total number of members in club.
3. Display the club members in reverse order.
4. Create 2 club and display concertinaed singly linked list.

Theory:

What is Linked list?

Linked list is a data structure that allows sequential access to the elements. A list lays out the sequence in a row, starting at the first element (called front / Head) and proceeding in successive order to the last element (called back / Last).

Each element in a list contains Links that identify both the next and the preceding item in the sequence. We focused on the fact that the Linked list provides efficient operations to insert and delete an element at any position in the sequence.

This was very important, because we needed to understand why a Linked list is sometimes used in an application rather than an array, which is another basic sequence data structure.

By the way, each elements of Linked list called node.

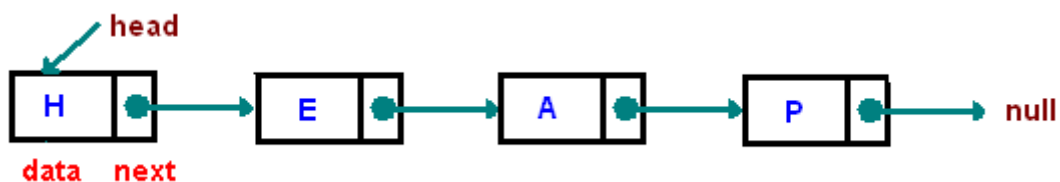
Linked List is a sequence of links which contains items. Each link contains a connection to another link. Linked list the second most used data structure after array. Following are important terms to understand the concepts of Linked List.

1. Link – Each Link of a linked list can store a data called an element.
2. Next – Each Link of a linked list contain a link to next link called Next.

Types of Linked List

1. **Singly linked list (SLL)**
2. **Doubly linked list (DLL)**
3. **Circular singly linked list (CSLL)**
4. **Circular doubly linked list (CDLL)**

Linked List Representation



Each element (we will call it a node) of a list is comprising of two items - the data and a reference to the next node.

The last node has a reference to null.

The entry point into a linked list is called the head of the list. It should be noted that head is not a separate node, but the reference to the first node. If the list is empty then the head is a null reference.

A linked list is a dynamic data structure. The number of nodes in a list is not fixed and can grow and shrink on demand.

Any application which has to deal with an unknown number of objects will need to use a linked list.

Advantages of Linked Lists

1. They are a dynamic in nature which allocates the memory when required.
2. Insertion and deletion operations can be easily implemented.
3. Stacks and queues can be easily executed.
4. Linked List reduces the access time.

Disadvantages of Linked Lists

1. The memory is wasted as pointers require extra memory for storage.
2. No element can be accessed randomly; it has to access each node sequentially.
3. Reverse Traversing is difficult in linked list.

Applications of Linked Lists

1. Linked lists are used to implement stacks, queues, graphs, etc.
2. Relational Database (Microsoft Access)
3. Hash tables and trees
4. Media player is example of doubly linked list
5. Multi-Player board game is one good example for having Circular linked list

Algorithm

/* Class Declaration for Singly linked list*/

Declare a class says Node for representing a node

Class Node

/* Public Section */

I. Declare following variable:

int data;

Node *next;

II. Stop

End Node class declaration

Declare a class say SLLinklist as follows

class SLLinklist

/* Public Section */

(1) Declare following procedures

a. void **CreateList()**; /* Procedure to create list or 'Pinnacle club' */

b. void **AddElement()**; /* Procedure to add element to list */

c. void **DisplayList()**; /* Procedure to display list */

d. void **ReverseList()**; /* Procedure to search elements of list */

e. void **MergeList()**; /* Procedure to merge two linked list */

f. **CreateList1()**; /* Procedure to create 2 linked list */

(2) Declare following variable:

Node *head, *last;

(3) Stop

end SLLinklist class declaration

/* MAIN FUNCTION */

procedure main ()

(1) Start

(2) Declare an integer variable *choice*

(3) Create instances of **SLLinklist** say L1, L2

(4) Create one *always true* loop e.g. while(1) //Start of while loop

(5) Display the following menu

SINGLY LINKED LIST OPERATIONS TO CREATE PINNACLE CLUB

1. **Create List** /* procedure CreateList() */

2. **Add Element** /* procedure AddElement() */

3. **Display List** /* procedure DisplayList() */
4. **Reverse Element in List** /* procedure SearchList() */
5. **MergeElement of two Lists** /* procedure DelElement() */
6. **Exit**

Select the Option:

(6) **Read** selected value in variable *choice* & perform the selected operation by calling the respective function.

(7) **Stop**

end procedure main

Function create a pinnacle club

CreateList ()

```
{
1. do
2. {
3. if(head==NULL)
4. {
    a. temp= new node;
    b. cout<<"\n Enter PRN And Name Of President:";
    c. cin>>temp->PRN>>temp->Name;
    d. temp->next=NULL;
    e. head=temp;
    f. temp= new node;
    g. cout<<"\n Enter PRN And Name Of Secretary:";
    h. cin>>temp->PRN>>temp->Name;
    i. temp->next=NULL;
    j. last=temp;
    k. head->next=last;
5. }
6. else
7. {
    a. temp1=head;
    b. while(temp1->next!=last)
    c. temp1=temp1->next;
    d. temp=new node;
    e. cout<<"\n Enter PRN And Name For Member:";
    f. cin>>temp->PRN>>temp->Name;
    g. temp->next=NULL;
    h. temp1->next=temp;
    i. temp->next=last;
8. }
9. cout<<"\n do you want to add new member";
10. cin>>op;
11. }while(op==1);
}
```

FUNCTION TO DISPLAY PINNACLE CLUB

DisplayList ()

```
{
```

```

1. Temp=head;
2. Print Name of precedent i.e.cout<<temp->PRN<<"\t\t"<<temp->Name<<"\n";
3. Temp=temp->next;
4. while(temp->next!=NULL) //Display all the members in club
5. {
    a. cout<<temp->PRN<<"\t\t"<<temp->Name<<"\n";
    b. temp=temp->next;
6. }
7. Print Name of Secretary i.e.cout<<temp->PRN<<"\t\t"<<temp->Name<<"\n";
}

```

Conclusion:

Understand the structure of singly linked, Implement traversal, insertion, deletion in a singly linked list; analyze the complexity of different operations in a singly linked list

QUESTIONS:

1. What is singly link list?
2. How to insert new element in linked list?
3. How to delete an element from linked list?
4. How to concatenate two lists?

Group C

Assignment 8

Problem Statement:

Write C++ program for storing binary number using doubly linked lists. Write functions

- To compute 1's and 2's complement
- Add two binary numbers

OBJECTIVES:

- To understand concept of OOP.
- To understand class structures for DLL in C++.
- To understand primitive operations on DLL.

Input:

- Accept Decimal number from user.

Output:

- compute 1's and 2's complement
- Show the addition of two binary numbers.

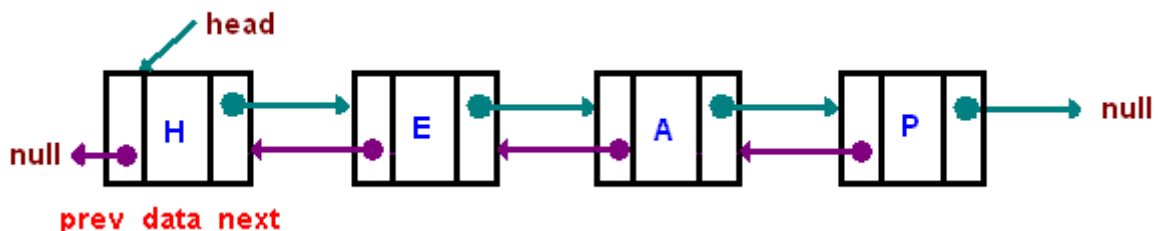
Theory:

What is Doubly Linked list?

A doubly-linked list is a linked data structure that consists of a set of sequentially linked records called nodes. Each node contains two fields, called links that are references to the previous and to the next node in the sequence of nodes.

The link fields of a doubly-linked list node are often called next and previous or forward and backward. The references stored in the link fields are usually implemented as pointers, but (as in any linked data structure) they may also be address offsets or indices into an array where the nodes live.

Doubly Linked List Representation



Each element (we will call it a node) of a list is comprising of three items - reference to the next node, data and a reference to the next node.

The first node previous and last node next has a reference to null.

The entry point into a linked list is called the head of the list. It should be noted that head is not a separate node, but the reference to the first node. If the list is empty then the head is a null reference.

A linked list is a dynamic data structure. The number of nodes in a list is not fixed and can grow and shrink on demand.

Any application which has to deal with an unknown number of objects will need to use a linked list.

Advantages of Doubly Linked list:

1. We can traverse in both directions i.e. from starting to end and as well as from end to starting.
2. It is easy to reverse the linked list.
3. If we are at a node, then we can go to any node. But in linear linked list, it is not possible to reach the previous node.

Disadvantages of Doubly Linked list:

1. It requires more space per space per node because one extra field is required for pointer to previous node.
2. Insertion and deletion take more time than linear linked list because more pointer operations are required than linear linked list.

Algorithm:

/* Class Declaration for Doubly linked list*/

Declare a class says Node for representing a node

Class Node

/* Public Section */

I. **Declare** following variable:

Node *prev;

int data;

Node *next;

II. **Stop**

End Node class declaration

Declare a class sayBinaryas follows

classBinary

/* Public Section */

(1) **Declare** following procedures

a. void **CreateList()**; /* Procedure to create list for binary numbers*/

b. void **DisplayList()**; /* Procedure to display binary number */

```

number */
d. void twos_compliment(); /* Procedure to perform twos_compliment of binary
number */
e. void Ones_compliment(); /*Procedure to perform ones compliment of binary
number */
f.Binary operator+(Binary); /* Procedure to perform addition of binary number */
(2) Declare following variable:

Node *head, *last;
(3) Stop

```

endSLLinklistclass declaration

```

/* MAIN FUNCTION */
proceduremain ()
(1) Start
(2) Declare an integer variable choice
(3) Create instances of DLinklists say L1, L2
(4) Create one always true loop e.g. while(1) //Start of while loop
(5) Display the following menu

```

SINGLY LINKED LIST OPERATIONS TO CREATE PINNACLE CLUB

```

1. Create List /* procedure CreateList() */
2. Display List /* procedure DisplayList() */
3. twos_compliment(); /* procedure twos_compliment() */
4. ones_compliment(); /* procedure ones_compliment() */
4. Addition of binary number /* procedure additionBinaryNumber() */
/* case 4:
    1. Read first decimal number
    2. Read second decimal number
    3. Create 2 object of class and call create list to convert both numbers into
       binary format
    4. Display converted binary numbers
    5. Create 3rd object of class and call addition function/*S = q+r; here s is the new
       object and q used to represent first linked list and r used to represent second
       linked list.Here + operator is used to called addition function */
    6. Display the addition of both binary numbers using 3rd object /*. s.display*/
*/

```

5. Exit

Select the Option:

```

(6) Read selected value in variable choice & perform the selected operation by calling the
respectivefunction.
(7) Stop

```

end procedure main

Algorithm to create a doubly linked list for binary number

CreateList ()

1. {


```

2.      Read decimal number from user say n
3.      while(n!=0)
4.      {
5.          rem=n%2;
6.          temp=new node;
7.          temp->next=NULL;
8.          temp->data=rem;
9.          temp->prev=NULL;
10.         if(head==NULL)
11.         {
12.             head=last=temp;
13.         }
14.         Else
15.         {
16.             last->next=temp;
17.             temp->prev=last;
18.             last=temp;
19.         }
20.         n=n/2;
21.     }
    }

```

Algorithm to display binary number

DisplayList()

```

1. {
2.     temp=last;
3.     while(temp!=NULL)
4.     {
5.         cout<<temp->data;
6.         temp=temp->prev;
7.     }
    }

```

Conclusion: study how to perform addition of binary numbers using Doubly Linked List

Oral Question:

1. Time complexity of DLL in insertion and deletion of node?
2. How to insert element in DLL?
3. How to delete element in DLL?
4. Difference between SLL and DLL?
5. Drawback of DLL?

Group D

Assignment 9

Problem Statement:

A palindrome is a string of character that's the same forward and backward. Typically, punctuation, capitalization, and spaces are ignored. For example, "Poor Dan is in a droop" is a palindrome, as can be seen by examining the characters "poor danisina droop" and observing that they are the same forward and backward. One way to check for a palindrome is to reverse the characters in the string and then compare with them the original-in a palindrome, the sequence will be identical. Write C++ program with functions-

- a) To print original string followed by reversed string using stack
- b) To check whether given string is palindrome or not

OBJECTIVE:

1. Understand Stack ADT.
2. Understand stack implementation using array.

Input:

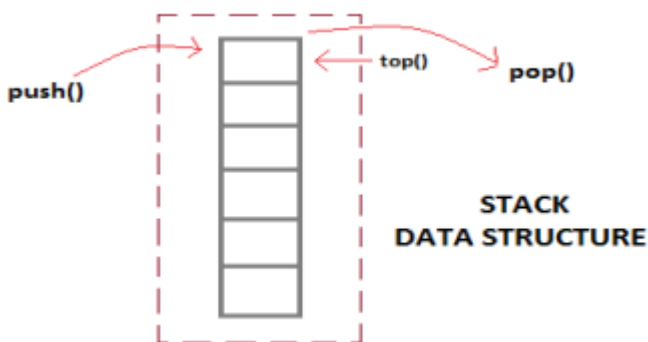
1. Accept string from user

Output:

1. Check string is palindrome or not.
2. Remove the space from string and the check for palindrome.
3. Print string in reverse order using stack.

Theory:

Stack is an abstract data type with a bounded (predefined) capacity. It is a simple data structure that allows adding and removing elements in a particular order. Every time an element is added, it goes on the top of the stack, the only element that can be removed is the element that was at the top of the stack, just like a pile of objects



Basic features of Stack

1. Stack is an ordered list of similar data type.
2. Stack is a LIFO structure. (Last in First out).
3. push() function is used to insert new elements into the Stack and pop() is used to delete an element from the stack. Both insertion and deletion are allowed at only one end of Stack called Top.

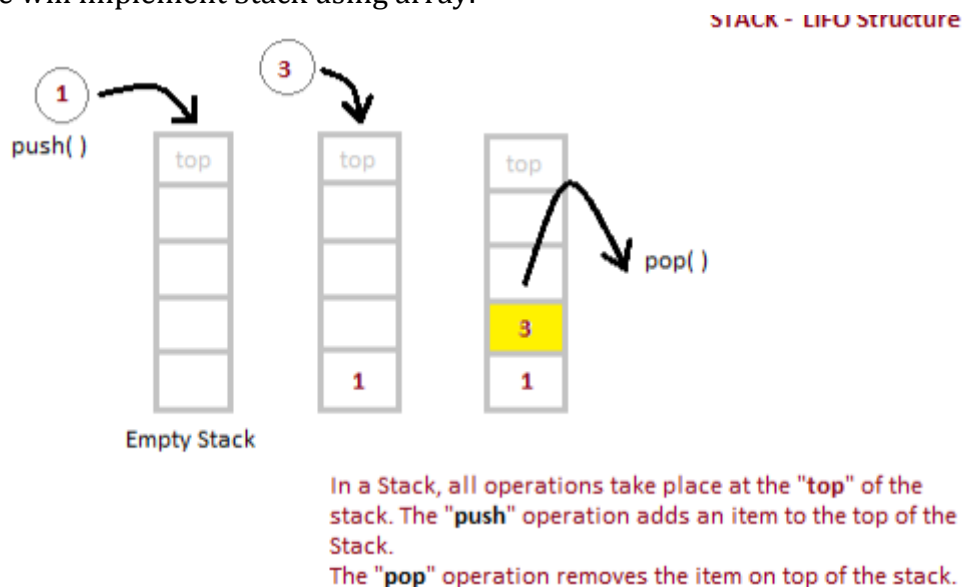
4. Stack is said to be in Overflow state when it is completely full and is said to be in Underflow state if it is completely empty.

Applications of Stack

The simplest application of a stack is to reverse a word. You push a given word to stack - letter by letter - and then pop letters from the stack. There are other uses also like :Parsing, Expression Conversion(Infix to Postfix, Postfix to Prefix etc) and many more.

Implementation of Stack

Stack can be easily implemented using an Array or a Linked List. Arrays are quick, but are limited in size and Linked List requires overhead to allocate, link, unlink and deallocate but is not limited in size. Here we will implement Stack using array.



Algorithm

Declare following variable globally i.e.i, j;
Declare max=100 as a constant using #define;

Classstack

/* private section*/

1. Declare following variable

2. Int top;

3. Char data[max]

/* Public Section */

(1) Declare following procedures

a. void push(char);

b. void pop();

c. void palindrome(char[]);

(2) Declare following variable:

Int length

(3) Stop

endstackclass declaration

```
/* MAIN FUNCTION */
```

```
proceduremain ()
```

```
(1) Start
```

```
(2) Declare an integer variable choice
```

```
(3) Create one always true loop e.g. while(1) //Start of while loop
```

```
(4) Display the following menu
```

```
STACK OPERATION
```

```
1. Check string is palindrome or not using stack.
```

```
2. Convert the string into lower case; remove the space and the check for palindrome.
```

```
3. Print string in reverse order using stack.
```

```
4. Exit
```

```
Select the Option:
```

```
(5) Read selected value in variable choice & perform the selected operation by calling the  
respective function.
```

```
(6) Stop
```

```
end procedure main
```

```
case 1: /* to check string is palindrome or not using stack*/
```

```
1. read the string from user
```

```
2. pass the string to the palindrome function
```

```
case 2: /* convert the string into lower case, remove the space from original  
string and the check for the palindrom */
```

```
1. read the string with white space from user using getline function
```

```
2. Convert the string into lower case, remove the space
```

```
3. For (i=0,j=0; str[i]!='\0';i++)
```

```
4. {
```

```
5. If(str[i]>=65 &&str[i]<=90)
```

```
6. {
```

```
7. Str1[j]=str[i]+32;
```

```
8. J++;
```

```
9. }
```

```
10. Else if(str[i]>=97 &&str[i]<=122)
```

```
11. {
```

```
12. Str1[j]=str[i];
```

```
13. J++;
```

```
14. }
```

```
15. }
```

```
16. Str1[j]='\0';
```

```
17. call palindrome functionpalindrome(str1);.
```

```
Case 3: /* print string in reverse order */
```

```
1. Read a string from user
```

2. Push string into stack
3. Pop the element and print the same

Analysis

Below mentioned are the time complexities for various operations that can be performed on the Stack data structure.

1. Push Operation : $O(1)$
2. Pop Operation : $O(1)$
3. Top Operation : $O(1)$
4. Search Operation : $O(n)$

Conclusion:

We learn ADT of stack along with the complexity analysis.

Oral Question:

1. What is stack?
2. What is a stack data structure, and how does it work?
3. Describe the main operations associated with a stack.
4. How does a stack follow the Last-In-First-Out (LIFO) principle? Provide an example.
5. What are the two primary actions performed on a stack?
6. How is a stack different from an array or a linked list?
7. Explain the concept of a "top" in the context of a stack.
8. What happens when you try to pop an element from an empty stack?
9. How do you check if a stack is empty or full?
10. Demonstrate how you would push elements "A," "B," and "C" onto a stack, and then pop two elements.
11. Can you implement a stack using an array or a linked list? Explain the advantages and disadvantages of each approach.

Group D

Assignment 10

Problem Statement:

Implement C++ program for expression conversion as infix to postfix and its evaluation using stack based on given conditions:

1. Operands and operator, both must be single character.
2. Input Postfix expression must be in a desired format.
3. Only '+', '-', '*' and '/' operators are expected.

OBJECTIVE:

1. Understand Stack ADT.
2. Understand stack application for Expression conversion and evaluation.

Input:

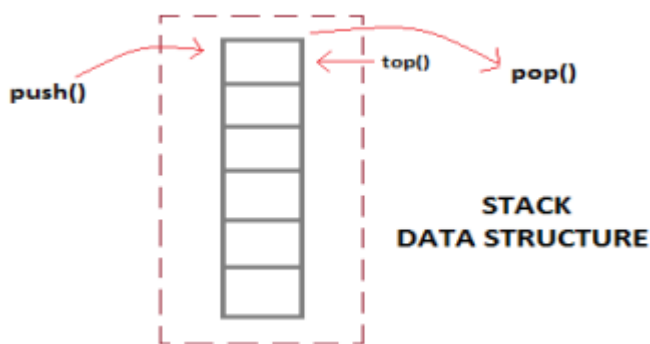
1. Accept infix expression from user

Output:

1. Display converted postfix expression
2. Display result of evaluated postfix form

Theory:

Stack is an abstract data type with a bounded (predefined) capacity. It is a simple data structure that allows adding and removing elements in a particular order. Every time an element is added, it goes on the top of the stack, the only element that can be removed is the element that was at the top of the stack, just like a pile of objects



Basic features of Stack

1. Stack is an ordered list of similar data type.
2. Stack is a LIFO structure. (Last in First out).
3. push() function is used to insert new elements into the Stack and pop() is used to delete an element from the stack. Both insertion and deletion are allowed at only one end of Stack called Top.
4. Stack is said to be in Overflow state when it is completely full and is said to be in Underflow state if it is completely empty.

Applications of Stack

The simplest application of a stack is to reverse a word. You push a given word to stack - letter by letter - and then pop letters from the stack. There are other uses also like : Parsing, Expression Conversion(Infix to Postfix, Postfix to Prefix etc) and many more.

Postfix notation is also known as Reverse Polish Notation (RPN) in which every operator follows all of its operands. This notation is parenthesis free. For e.g, (A + B) is expressed as AB+ in postfix notation. We use the following straight-forward algorithm to convert infix expression to a postfix expression:-

Algorithm

1. Declare following variable globally i.e.i, j;
2. Declare max=100 as a constant using #define

Classstack

/* private section*/

1. Declare following variable
2. Int top;
3. Char data[max]

/* Public Section */

(1) Declare following procedures

- a. void push(char);
- b. void pop();
- c. void infix_postfix;

(2) Declare following variable:

Int length

(3) Stop

endstackclass declaration

Algorithm to convert infix expression to the postfix notation.

- 1) Scan the given expression from left to right.
- 2) First operator seen is simply pushed onto stack.
- 3) If we see an operand, append it to the postfix expression.
- 4) If we see an operator (x), pop off all the operators which are of lower precedence than 'x' and append them to the postfix expression. Then, push the operator 'x' onto stack.
- 5) If we see an opening parenthesis, simply push it onto stack.
- 6) If we see a closing parenthesis, pop off all elements from stack till opening parenthesis and append them to postfix expression except the opening & closing parenthesis.
- 7) Finally, pop off all the elements (operators) from stack till it's empty and append them to postfix expression.

Algorithm to evaluate a postfix expression.

- 1) Scan the expression from left to right.
- 2) If an operand is seen, push it onto stack.
- 3) If an operator is seen, pop of top two elements (operands) [x & y] from stack and perform $z = x \text{ operand } y$. Push z onto stack.
- 4) Repeat steps (2) & (3) till scanning is over.

Analysis

Below mentioned are the time complexities for various operations that can be performed on the Stack data structure.

1. Push Operation : $O(1)$
2. Pop Operation : $O(1)$
3. Top Operation : $O(1)$
4. Search Operation : $O(n)$

Conclusion:

We learn ADT of stack, Expression conversion using stack along with the complexity analysis.

Oral Question:

1. What is the space complexity of a stack?
2. Describe real-world scenarios where a stack data structure is useful.
3. How can a stack be used to evaluate arithmetic expressions?
4. Explain the role of a call stack in programming languages and function calls.
5. How do you prevent a stack overflow, and what can cause it?

Group E

Assignment 11

Problem Statement:

Queues are frequently used in computer programming, and a typical example is the creation of a job queue by an operating system. If the operating system does not use priorities, then the jobs are processed in the order they enter the system. Write C++ program for simulating job queue. Write functions to add job and delete job from queue

Objectives:-

1. To understand concept of queue.
2. To study the different types of queues.
3. To study the job scheduling problem as queue application.

Input:-

1. Accept the data from user

Output:-

1. To implement primitive operations on queue.

Theory:-

A queue can be defined as an ordered list which enables insert operations to be performed at one end called REAR and delete operations to be performed at another end called FRONT. Queue is referred to be as First in First Out list. Queue is an abstract data structure, somewhat similar to Stack. In contrast to Queue, queue is opened at both ends.



Applications of Queue

Due to the fact that queue performs actions on first in first out basis which is quite fair for the ordering of actions. There are various applications of queues discussed as below.

1. Queues are widely used as waiting lists for a single shared resource like printer, disk, CPU.
2. Queues are used in asynchronous transfer of data (where data is not being transferred at the same rate between two processes) for eg. pipes, file IO, sockets.
3. Queues are used as buffers in most of the applications like MP3 media player, CD player, etc.

5. Queues are used to maintain the play list in media players in order to add and remove the songs from the play-list.
6. Queues are used in operating systems for handling interrupts.

Operations on Queue:

Mainly the following six basic operations are performed on queue:

1. Enqueue: Adds an item to the queue. If the queue is full, then it is said to be an Overflow condition.
2. Dequeue: Removes an item from the queue. The items are removed in the same order in which they are inserted. If the queue is empty, then it is said to be an Underflow condition.
3. getFront: Get the front item from queue.
4. getRear: Get the last item from queue.
5. Queue overflow (isfull): When the Queue is completely full, then it shows the overflow condition.
6. Queue underflow (isempty): When the Queue is empty, i.e., no elements are in the Queue then it throws the underflow condition.

Algorithm

Queue Operations using Array

Queue data structure using array can be implemented as follows.

Before we implement actual operations, first follow the below steps to create an empty queue.

- **Step 1** - Include all the **header files** which are used in the program and define a constant '**SIZE**' with specific value.
- **Step 2** - Declare all the **user defined functions** which are used in queue implementation.
- **Step 3** - Create a one dimensional array with above defined SIZE (**int queue[SIZE]**)
- **Step 4** - Define two integer variables '**front**' and '**rear**' and initialize both with '**-1**'. (**int front = -1, rear = -1**)
- **Step 5** - Then implement main method by displaying menu of operations list and make suitable function calls to perform operation selected by the user on queue.

enQueue(value) - Inserting value into the queue

In a queue data structure, enQueue() is a function used to insert a new element into the queue. In a queue, the new element is always inserted at **rear** position. The enQueue() function takes one integer value as a parameter and inserts that value into the queue. We can use the following steps to insert an element into the queue...

- **Step 1** - Check whether **queue** is **FULL**. (**rear == SIZE-1**)
- **Step 2** - If it is **FULL**, then display "**Queue is FULL!!! Insertion is not possible!!!**" and terminate the function.
- **Step 3** - If it is **NOT FULL**, then increment **rear** value by one (**rear++**) and set **queue[rear] = value**.

deQueue() - Deleting a value from the Queue

In a queue data structure, deQueue() is a function used to delete an element from the queue. In a queue, the element is always deleted from **front** position. The deQueue() function does not take any value as parameter. We can use the following steps to delete an element from the queue...

- **Step 1** - Check whether **queue** is **EMPTY**. (**front == rear**)

- **Step 2** - If it is **EMPTY**, then display "**Queue is EMPTY!!! Deletion is not possible!!!**" and terminate the function.
- **Step 3** - If it is **NOT EMPTY**, then increment the **front** value by one (**front ++**). Then display **queue[front]** as deleted element. Then check whether both **front** and **rear** are equal (**front == rear**), if it **TRUE**, then set both **front** and **rear** to '-1' (**front = rear = -1**).

display() - Displays the elements of a Queue

We can use the following steps to display the elements of a queue...

- **Step 1** - Check whether **queue** is **EMPTY**. (**front == rear**)
- **Step 2** - If it is **EMPTY**, then display "**Queue is EMPTY!!!**" and terminate the function.
- **Step 3** - If it is **NOT EMPTY**, then define an integer variable '**i**' and set '**i = front+1**'.
- **Step 4** - Display '**queue[i]**' value and increment '**i**' value by one (**i++**). Repeat the same until '**i**' value reaches to **rear** (**i <= rear**)

Complexity Analysis of Queue Operations:

1. Enqueue: O(1)
2. Dequeue: O(1)

Conclusion: Thus, we have successfully implemented queue using array.

Oral Question:

1. What is a queue data structure, and how does it work?
2. Describe the main operations associated with a queue.
3. How does a queue follow the First-In-First-Out (FIFO) principle? Provide an example.
4. What are the two primary actions performed on a queue?
5. How is a queue different from a stack?
6. Explain the concept of "front" and "rear" in the context of a queue.
7. What happens when you try to dequeue an element from an empty queue?
8. How do you check if a queue is empty or full?
- 9.

Group E

Assignment 12

Problem Statement:

A double-ended queue (deque) is a linear list in which additions and deletions may be made at either end. Obtain a data representation mapping a deque into a one-dimensional array. Write C++ program to simulate deque with functions to add and delete elements from either end of the deque.

Objectives:-

1. To understand concept of double ended queue.
2. To study the different types of double ended queues.

Input:-

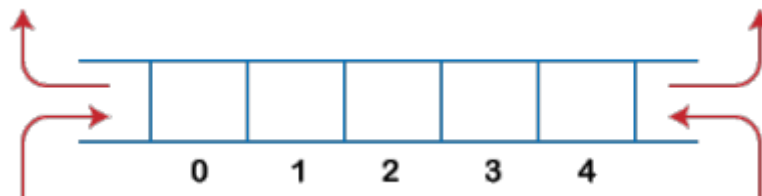
1. Accept the data from user

Output:-

1. To implement primitive operations on double ended queue.

Theory:

A double-ended queue, often abbreviated as "deque," is a data structure that allows elements to be inserted and removed from both ends. It combines the features of both stacks and queues, providing efficient operations at both ends.



Here are the main operations associated with a double-ended queue:

1. Insert at front
2. Delete from end
3. Insert at rear
4. Delete from rear

EnqueueFront (Insertion at the front):

To insert an element at the front of the deque, first, check if the deque is full (if it has a maximum size).

If the deque is not full, decrement the front pointer and add the new element at the front position.

If the deque is full, it is considered an overflow condition, and the element cannot be added.

EnqueueRear (Insertion at the rear):

To insert an element at the rear of the deque, first, check if the deque is full (if it has a maximum size).

If the deque is not full, increment the rear pointer and add the new element at the rear position.

If the deque is full, it is considered an overflow condition, and the element cannot be added.

DequeFront (Deletion from the front):

To remove an element from the front of the deque, first, check if the deque is empty. If the deque is not empty, remove the element at the front position and increment the front pointer. If the deque is empty, it is considered an underflow condition, and there are no elements to remove.

DequeRear (Deletion from the rear):

To remove an element from the rear of the deque, first, check if the deque is empty. If the deque is not empty, remove the element at the rear position and decrement the rear pointer. If the deque is empty, it is considered an underflow condition, and there are no elements to remove.

IsEmpty:

This operation checks if the deque is empty by comparing the front and rear pointers. If they are equal, the deque is empty.

IsFull:

This operation checks if the deque is full by comparing the rear pointer with the maximum size of the deque. If the rear pointer is equal to the size minus one, the deque is full.

These operations make a double-ended queue a versatile data structure, suitable for a wide range of applications. It allows elements to be added or removed from both ends efficiently, enabling scenarios where elements are processed from either end as needed. Double-ended queues are particularly useful in scenarios that require efficient insertion and deletion at both ends, such as managing sliding windows in algorithms and implementing data structures like dequeued queues (also known as "priority dequeues").

Algorithms:-**Insert Elements at Rear end**

- a) First we check deque if Full or Not
 - b) IF $\text{Rear} == \text{Size} - 1$
then reinitialize $\text{Rear} = 0$;
 - Else
increment Rear by '1'
insert current key into $\text{Arr}[\text{rear}] = \text{key}$
- Front remain same.

Insert Elements at Front end

- a) First we check deque if Full or Not
 - b) IF $\text{Front} == 0$ || initial position, move Front to points last index of array
 $\text{front} = \text{size} - 1$
 - Else
decremented front by '1' and push current key into $\text{Arr}[\text{Front}] = \text{key}$
- Rear remains same.

Delete Element from Rear end

- a) first Check deque is Empty or Not
- b) If deque has only one element

```

    front = -1 ; rear = -1 ;
Else IF Rear points to the first index of array
    it's means we have to move rear to points last index [ now first inserted element at
front
    end become rear end ]
    rear = size-1 ;
Else || decrease rear by '1'
    rear = rear-1;

```

Delete Element From Front end

```

a) first Check deque is Empty or Not
b) If deque has only one element
    front = -1 ; rear = -1 ;
Else IF front points to the last index of the array
    it's means we have no more elements in array so we move front to points first index
of array
    front = 0 ;
Else || increment Front by '1'
    front = front+1;

```

isEmpty() :

If front is equals to -1 the Deque is empty, else it's not.

isFull() :

If $(\text{rear} + 1) \% n$ equals to the front then the Deque is full, else it's not. Here n is that the most size of Deque.

Conclusion: Hence we have learned how to implement Double Ended Queue and perform various operations on it.

Oral Question:

1. What is a double-ended queue (deque) data structure, and how does it differ from a regular queue?
2. How can elements be inserted into a deque from both ends? Provide examples of enqueue operations at the front and rear.
3. Explain how deque maintains the First-In-First-Out (FIFO) order for elements inserted from the front and the rear.
4. Describe the main operations associated with a deque.
5. What happens when you try to dequeue an element from an empty deque? How is this different from dequeuing from an empty regular queue?
6. How do you handle overflow and underflow conditions in a deque?
7. Can you illustrate how a deque can be implemented using an array or a linked list?
8. What are the advantages of using a deque over other linear data structures, such as stacks and regular queues?

Group E
Assignment 13

Problem Statement:

Pizza parlor accepting maximum M orders. Orders are served in first come first served basis. Order once placed cannot be cancelled. Write C++ program to simulate the system using circular queue using array.

Objectives:-

1. To understand concept of circular queue.

Input:-

1. Accept the data from user

Output:-

1. To implement primitive operations on circular queue

Theory:

In a normal Queue Data Structure, we can insert elements until queue becomes full. But once if queue becomes full, we can't insert the next element until all the elements are deleted from the queue. For example consider the queue below... After inserting all the elements into the queue

Queue is Full



Now consider the following situation after deleting three elements from the queue...

Queue is Full (Even three elements are deleted)

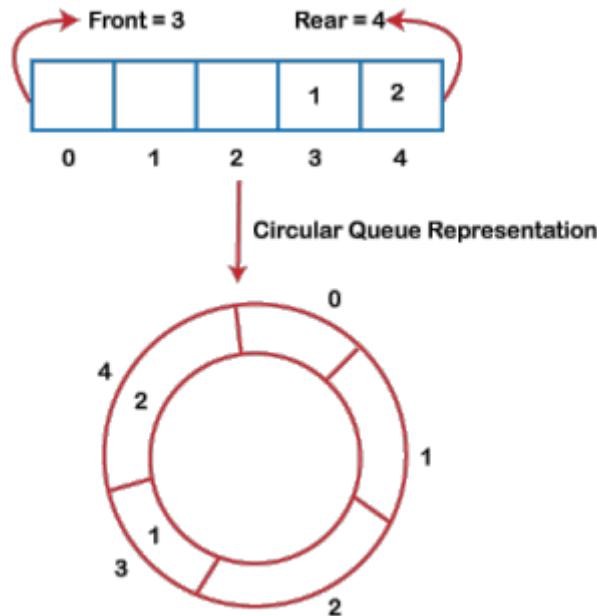


This situation also says that Queue is full and we can't insert the new element because, „rear“ is still at last position. In above situation, even though we have empty positions in the queue we can't make use of them to insert new element. This is the major problem in normal queue data structure. To overcome this problem we use circular queue data structure.

What is a Circular Queue?

A Circular Queue can be defined as follows...

Circular Queue is a linear data structure in which the operations are performed based on FIFO (First in First Out) principle and the last position is connected back to the first position to make a circle. Graphical representation of a circular queue is as follows



Algorithm of Circular Queue

To implement a circular queue data structure using an array, we first perform the following steps before we implement actual operations.

- **Step 1** - Include all the **header files** which are used in the program and define a constant '**SIZE**' with specific value.
- **Step 2** - Declare all **user defined functions** used in circular queue implementation.
- **Step 3** - Create a one dimensional array with above defined SIZE (**int cQueue[SIZE]**)
- **Step 4** - Define two integer variables '**front**' and '**rear**' and initialize both with '**-1**'. (**int front = -1, rear = -1**)
- **Step 5** - Implement main method by displaying menu of operations list and make suitable function calls to perform operation selected by the user on circular queue.

enQueue(value) - Inserting value into the Circular Queue

In a circular queue, enQueue() is a function which is used to insert an element into the circular queue. In a circular queue, the new element is always inserted at **rear** position. The enQueue() function takes one integer value as parameter and inserts that value into the circular queue. We can use the following steps to insert an element into the circular queue...

- **Step 1** - Check whether **queue** is **FULL**. **((rear == SIZE-1 && front == 0) || (front == rear+1))**
- **Step 2** - If it is **FULL**, then display "**Queue is FULL!!! Insertion is not possible!!!**" and terminate the function.
- **Step 3** - If it is **NOT FULL**, then check **rear == SIZE - 1 && front != 0** if it is **TRUE**, then set **rear = -1**.
- **Step 4** - Increment **rear** value by one (**rear++**), set **queue[rear] = value** and check '**front == -1**' if it is **TRUE**, then set **front = 0**.

deQueue() - Deleting a value from the Circular Queue

In a circular queue, deQueue() is a function used to delete an element from the circular queue. In a circular queue, the element is always deleted from **front** position. The deQueue() function doesn't take any value as a parameter. We can use the following steps to delete an element from the circular queue...

- **Step 1** - Check whether **queue** is **EMPTY**. (**front == -1 && rear == -1**)
- **Step 2** - If it is **EMPTY**, then display "**Queue is EMPTY!!! Deletion is not possible!!!**" and terminate the function.
- **Step 3** - If it is **NOT EMPTY**, then display **queue[front]** as deleted element and increment the **front** value by one (**front ++**). Then check whether **front == SIZE**, if it is **TRUE**, then set **front = 0**. Then check whether both **front - 1** and **rear** are equal (**front - 1 == rear**), if it **TRUE**, then set both **front** and **rear** to **-1** (**front = rear = -1**).

display() - Displays the elements of a Circular Queue

We can use the following steps to display the elements of a circular queue...

- **Step 1** - Check whether **queue** is **EMPTY**. (**front == -1**)
- **Step 2** - If it is **EMPTY**, then display "**Queue is EMPTY!!!**" and terminate the function.
- **Step 3** - If it is **NOT EMPTY**, then define an integer variable **'i'** and set **'i = front'**.
- **Step 4** - Check whether **'front <= rear'**, if it is **TRUE**, then display **'queue[i]'** value and increment **'i'** value by one (**i++**). Repeat the same until **'i <= rear'** becomes **FALSE**.
- **Step 5** - If **'front <= rear'** is **FALSE**, then display **'queue[i]'** value and increment **'i'** value by one (**i++**). Repeat the same until **'i <= SIZE - 1'** becomes **FALSE**.
- **Step 6** - Set **i** to **0**.
- **Step 7** - Again display **'cQueue[i]'** value and increment **i** value by one (**i++**). Repeat the same until **'i <= rear'** becomes **FALSE**.

Conclusion: Hence we have learned how to implement Circular Queue and perform various operations on it.

Oral Questions:

1. What is a circular queue data structure, and how does it differ from a regular queue?
2. How is a circular queue implemented to achieve the circular behavior?
3. Explain the advantages of using a circular queue over a regular linear queue.
4. Describe the main operations associated with a circular queue.
5. How do you handle overflow and underflow conditions in a circular queue?
6. What happens when the front and rear pointers meet during enqueue and dequeue operations?
7. Can you illustrate how a circular queue can be implemented using an array?
8. Discuss the time complexity of the main circular queue operations (enqueue, dequeue, isEmpty, isFull) when implemented using an array.